

Evidence-Backed Release Assurance for Autonomous Agent Deployments: Cryptographic Attestation, Fail-Closed Gates, and Tamper-Resilient Audit Infrastructure

Anonymous Submission
Submission ID: EVI-227

Abstract

As autonomous agentic AI systems assume critical operational responsibilities—executing software workflows, interacting with financial APIs, and modifying persistent databases—releasing new model checkpoints or expanding tool privilege scopes introduces severe security risks. Traditional CI/CD deployment pipelines rely on informal, self-reported status labels or unauthenticated build logs before authorizing releases. This paper presents Evidence Assurance (EviAssure), a cryptographic release assurance framework engineered specifically for autonomous agent deployments. EviAssure’s core contribution is *evidence-bound fail-closed release authorization*: the release decision is a deterministic, auditable function of cryptographically attested execution evidence under a pinned policy version, with freshness, replay, and revocation enforced over the same signed payload. The system contributes (1) an Ed25519 and SHA-256 Merkle tree execution trace attestation engine that seals agent action histories into immutable evidence packs carrying signed trace counts and depth-bound inclusion proofs; (2) an in-toto v0.2 / SLSA v1.0 compatible `EvidenceBundle` envelope with nonces, timestamps, multi-signature threshold gating, and privacy-preserving salted trace parameter blinding; (3) a fail-closed `ReleasePolicyEngine` enforcing multi-factor release conditions, KMS key ARN verification, and key revocation list (CRL) validation; and (4) a per-leaf forensic inspection layer that surfaces semantic misbehavior that an integrity-preserving gate cannot reject. Empirical benchmarks demonstrate that EviAssure achieves a 100.0

1 Introduction

Autonomous software agents driven by Large Language Models (LLMs) operate dynamically across complex tool environments, API endpoints, and persistent memory stores [10, 32, 33]. Unlike static software artifacts whose behaviors are deterministically governed by compiled code, autonomous agents exhibit non-deterministic reasoning patterns, dynamic

tool execution flows, and adaptive multi-step decision sequences. When deploying agent updates, expanding tool privilege scopes, or promoting agentic pipelines to production, organization-wide security depends upon verifying that safety invariants, authorization constraints, and regression test suites have been rigorously validated [10].

Existing software delivery and release pipelines suffer from a fundamental trust defect: traditional deployment gates evaluate unauthenticated status signals (such as process exit codes, plain text console logs, or un-signed JSON build status files) generated by intermediate runner environments. An attacker who gains control over an intermediate build step, manipulates agent memory traces, or exploits prompt injection vectors can easily forge release approval signals, bypassing security verification entirely [7, 24, 29, 31].

Empirical studies of the open-source ecosystem confirm both the prevalence and the practical exploitability of these compromise paths: taxonomies of documented supply-chain attacks now enumerate hundreds of real incidents [16, 22], large-scale measurement work shows package-manager compromise to be routine in interpreted-language ecosystems [9], and multi-repository update systems require dedicated defenses [20].

To address these critical vulnerabilities, EviAssure introduces an evidence-backed release assurance architecture that replaces informal release signals with cryptographically authenticated Evidence Bundles backed by Merkle inclusion proofs, timestamp freshness windows, Ed25519 digital signatures, KMS key ARNs, and SLSA v1.0 provenance envelopes.

1.1 Key Contributions

The primary technical and systems contributions of this work include:

1. **Evidence-Bound Fail-Closed Release Authorization:** The system-level contribution is not a new attestation format but an authorization layer: `ReleasePolicyEngine` outputs a deterministic, reproducible verdict that is a function of attested evidence under a pinned policy ver-

sion, so an APPROVED release carries its own security proof and the verdict is auditable post-hoc by recomputation.

2. **Cryptographic Trace Attestation Engine:** An Ed25519 public-key signature scheme [4, 6] coupled with a SHA-256 Merkle tree aggregation protocol that condenses up to $N = 1,000,000$ agent execution traces into one immutable 32-byte root digest, with the signed trace count binding the root to exactly the submitted action set (completeness-carrying binding).
3. **Multi-Signature Threshold Gating:** Threshold verification policies (at least K of M authorized signatures) before production deployment authorization.
4. **Salted Trace Parameter Blinding:** A salted HMAC blinding mode (`blind_payload`) using keyed hashes [14] that domain-separates record digests with a per-deployment salt so the same recorded action cannot be correlated across contexts, while preserving public Merkle auditability. Raw prompt text, API inputs, and PII are excluded at the collector—evidence bundles carry digests only, never raw payload strings (Section 3.2).
5. **Browser DOM & Visual Attestation:** Execution trace attestation extended to browser UI actions, DOM state hashes ($\text{SHA256}(\text{DOM}_{\text{serialized}})$), and screenshot render-reference digests.
6. **System-Level and Primitive-Level Security Proofs:** A sequence-of-games proof reducing tamper-resistance to EUF-CMA and hash collision resistance [3], together with a system-level *Release Authorization Soundness* theorem (Section 4.2) stating what an APPROVED verdict means and the exact trust boundaries (evidence collector, key registry, nonce state) on which that meaning rests.
7. **Two-Layer Evidence Model:** Separation of cryptographic *integrity* (enforced by the gate) from per-leaf *semantic inspection* (surfacing recorded actions that should not have occurred), evaluated at corpus scale over 1,050 agent profiles.
8. **Corpus & Forensic Infrastructure:** A 1,050-profile multi-architecture agent trace corpus (`corpus/agent_trace_corpus.json`) with 50 anomaly profiles spanning 10 anomaly types, and a CLI forensic inspection tool (`scripts/audit_trace_forensics.py`).
9. **Empirical Evaluation:** 100.0% block rate across 12 tamper vectors versus 0.0% (CI exit-code), 25.0% (executed OPA schema gate), and 8.3% (Sigstore-style signature-presence gate), with all timed measurements reported as means over 5 runs.

Positioning. in-toto provides signed, tamper-evident attestations of supply-chain steps—including a vetted runtime-trace predicate [12]—and SLSA builds graduated release provenance on top [29, 31]; in 2026 the envelope ecosystem extended to AI agents via the `agent-decision/v0.1` predicate

RFC [2], the Agent Provenance Attestation Standard [1], and the Agent Auditability Standard AAS-1 [13]. These efforts define *record formats*; none defines an *authorization layer*: a release gate whose verdict is a deterministic function of attested evidence under a pinned policy version, with freshness, replay, and revocation enforced over the same signed payload. Record formats and authorization layers compose rather than compete—EviAssure’s `EvidenceBundle` is an in-toto v0.2 / SLSA v1.0 compatible envelope, so any predicate that fits a Statement can ride on the same evidence pack. What EviAssure adds that no predicate alone provides is (i) completeness-carrying trace binding via the signed trace count, (ii) freshness and replay protection as *integral gate properties* rather than attestation fields a verifier may or may not enforce, (iii) evidence-bound authorization whose decision is reproducible and auditable, and (iv) a two-layer forensic model separating cryptographic integrity from per-leaf semantic inspection. Section 7.7 positions EviAssure against these developments in detail.

2 System Architecture and Threat Model

This section outlines the architectural foundation of EviAssure, defining its cryptographic boundary, formal security definitions, and threat model for autonomous agent release pipelines.

2.1 Hardware Enclave & KMS Boundary

To defend against local runner compromise, EviAssure roots signing authority within a Hardware Security Module (HSM) or Cloud KMS boundary (`kms://aws/arn:aws:kms:us-east-1:000000000000:key/usenix-release-gate`). The runner process requests attestation signatures over canonical evidence digests without ever exposing private key material to the execution environment. The research prototype signs locally with a pinned demo key; production deployments root the private half in the KMS/HSM, and the gate enforces the ARN boundary (`kms_key_arn_pattern`) at verification time regardless of where signing occurred.

2.2 Formal Security Definitions

Definition 1 (Unforgeability under Chosen Message Attack (EUF-CMA)). *An Evidence Assurance Scheme $\Sigma = (\text{KeyGen}, \text{Package}, \text{Verify})$ is EUF-CMA secure if for all probabilistic polynomial-time adversaries $\mathcal{A}$, the probability that $\mathcal{A}$ outputs a valid pair (E^*, σ^*) such that $\text{Verify}(PK, E^*, \sigma^*) = 1$ without E^* having been signed by an honest party holding SK is negligible in security parameter λ :*

$$\Pr[\text{Exp}_{\Sigma, \mathcal{A}}^{\text{euf-cma}}(\lambda) = 1] \leq \text{negl}(\lambda) \quad (1)$$

Definition 2 (Merkle Trace Inclusion Soundness). *For root R_M committed together with its leaf count N , and leaf H_i , a Merkle proof π_i is sound if no computationally bounded adversary can construct π'_i for any $H'_i \neq H_i$ — including internal node digests of the same tree — such that $\text{VerifyMerkle}(H'_i, \pi'_i, R_M, \lceil \log_2 N \rceil) = 1$, where verification accepts only proof paths of the committed tree depth.*

Definition 3 (Salted Trace Parameter Blinding Indistinguishability). *A blinded trace record T_{blinded} produced with uniform salt $S \leftarrow \{0, 1\}^\lambda$ satisfies computational indistinguishability: no PPT adversary $\mathcal{A}$ can distinguish T_{blinded} from a uniformly random bitstring of equal length without knowledge of S .*

Definition 4 (Evidence Integrity). *An evidence pack (R_M, σ, N) comprising a Merkle root R_M , signature σ , and signed trace count N provides evidence integrity if no probabilistic polynomial-time adversary can produce any $(R'_M, \sigma', N') \neq (R_M, \sigma, N)$ that verifies; equivalently, nothing that was recorded can be modified, reordered, inserted, or removed without detection. Integrity is a property of the signed record and is guaranteed by Theorem 1 below.*

Definition 5 (Trace Completeness). *Let E be an agent execution and let C be the evidence collector that observes E . A recorded trace set $T = \{T_1, \dots, T_N\}$ is complete with respect to E if every action of E that C observed is represented by exactly one T_i . Completeness is a property of the collector, not of the Merkle tree: the tree attests exactly the leaves it was given, and no downstream check can distinguish an action that was never recorded from one that was honestly absent.*

2.3 Threat Model, Trust Boundaries, and Adversary Capabilities

The security model considers a powerful adversary $\mathcal{A}$ attempting to deploy an unapproved, regression-prone, or malicious agent update to production environments [8, 25, 26]. The adversary $\mathcal{A}$ may control intermediate runner environments, network transit layers, or build scripts.

Trust Model Boundaries. EviAssure explicitly assumes that the underlying cryptographic primitives (Ed25519 and SHA-256) remain computationally intractable for PPT adversaries, and that the root Cloud KMS / HSM service faithfully enforces identity access control over private signing keys. Physical hardware side-channel attacks on HSM microchips, total subversion of the cloud provider’s root KMS master keys, and host OS kernel hypervisor rootkits are explicitly considered out-of-scope for the release gate layer.

We categorize $\mathcal{A}$ ’s in-scope capabilities into five adversary capability classes:

- **$\mathcal{A}_1$ (Runner & Memory Trace Tampering):** Modifying in-memory agent execution logs, altering action output hashes, or dropping failed execution steps to present

a clean trace history. In an agentic setting, an attacker attempting to deploy a compromised model update might suppress execution traces recording unauthorized system shell commands or SQL table drop queries.

- **$\mathcal{A}_2$ (Network & Signature Forgery):** Injecting forged cryptographic signatures, stripping signature fields, or signing evidence payloads with unauthorized key pairs. An adversary controlling a CI builder might strip Ed25519 signatures or sign payloads using arbitrary self-generated keys.
- **$\mathcal{A}_3$ (Trace Replay & Time Skew):** Replaying historical, previously approved evidence bundles from earlier release cycles or injecting post-dated timestamps into the future to bypass validity windows. An attacker who cannot forge signatures might replay an old, benign evidence pack from a previous release to authorize a malicious deployment.
- **$\mathcal{A}_4$ (Key Revocation & Governance Bypass):** Using revoked key IDs listed in the Key Revocation List (CRL) or attempting KMS ARN parameter confusion. If an organization revokes a key following developer credential leaks, $\mathcal{A}$ attempts to use the revoked key before CRL updates propagate.
- **$\mathcal{A}_5$ (Serialization & Key Malleability Injection):** Injecting uncanonicalized JSON key fields or duplicate dictionary keys to exploit parser discrepancies between packager and verifier. By injecting duplicate JSON dictionary keys or non-standard white-spacing, $\mathcal{A}$ attempts to cause signature verification to succeed over one view while policy parsing evaluates another.

The release gate enforces a strict zero-trust fail-closed guarantee: any evidence payload matching any of these attack vectors is rejected with exit code 1 (BLOCKED).

Completeness boundary. The distinction between Definition 4 (evidence integrity) and Definition 5 (trace completeness) scopes every security claim in this paper. The cryptographic layer guarantees *integrity of whatever was recorded*: given the recorded sequence $A \rightarrow B \rightarrow D \rightarrow E$, no adversary can silently substitute $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$ or drop a recorded leaf. It cannot prove that an action executed before collection—say C —was recorded at all; a Merkle tree attests its leaves, not the world. EviAssure therefore scopes its guarantees to the *collection boundary*: the evidence collector is assumed to observe and record every action of the release execution, and Theorem 2 states this assumption explicitly. An adversary that compromises the recording path *before* collection can omit actions that no signature or Merkle check can expose—the same boundary every attested-execution system faces, and one the 2026 provenance literature reaches independently (APAS requires external witnesses at its top level; AAS-1 detects omission only *between* records one issuer produced) [1, 13]. Residual risk is mitigated by col-

lecting evidence at the outermost observable boundary (the tool-execution or gateway layer) and by semantic inspection (Section 6.9), which surfaces recorded actions that should not have occurred. Actions that never reach the collector are out of scope for any release-attestation system.

3 Cryptographic Evidence Attestation

EviAssure establishes verifiable release claims by packaging agent action histories into cryptographically attested evidence bundles. This section describes the Merkle tree aggregation protocol, browser DOM attestation, trace parameter blinding, and SLSA v1.0 / in-toto predicate envelopes.

3.1 Execution Trace & Browser UI Attestation Schema

Each execution trace record T_i captures structured action metadata across autonomous agent steps. For API and backend tool calls, T_i records:

$$T_i = (\text{id}_i, \text{agent}_i, \text{action}_i, \text{status}_i, \text{duration}_i, H_{\text{out},i}) \quad (2)$$

For browser and UI agent steps, EviAssure extends attestation to capture DOM state hashes (H_{DOM}) and visual screenshot render digests (H_{screen}):

$$T_{\text{UI},i} = (\text{id}_i, \text{agent}_i, \text{action}_i, \text{url}_i, \text{elem}_i, H_{\text{DOM},i}, H_{\text{screen},i}) \quad (3)$$

H_{DOM} is computed over the serialized DOM state of the rendered page, $H_{\text{DOM},i} = \text{SHA256}(\text{DOM}_{\text{serialized}})$: the specimen serializes the rendered document deterministically before hashing, so identical page states yield identical digests while any mutation (e.g., an injected modal dialog or altered form control) changes the digest. Each trace record produces a canonical leaf digest $H_i = \text{SHA256}(T_i.\text{to_hash}())$. The canonical leaf serialization commits *every* schema field—identifier, agent, action, status, duration, and output digest—so modifying any recorded field after signing, including the timing `duration_ms`, invalidates the recomputed root and is rejected by the gate.

3.2 Salted Trace Parameter Blinding Protocol

To protect proprietary prompt text, customer PII, or sensitive API parameters during public audit, EviAssure implements a keyed HMAC blinding mode (`blind_payload`). The binding assumption is stated first: evidence bundles never contain raw payload strings—the trace records store only digests (`output_hash`) and the `raw_payload` field is excluded from serialization—so secrecy rests on excluding raw text at the collector, and blinding does not (and need not) conceal data that is absent from the bundle. What blinding adds

is *domain separation*: given the trace’s output digest P_i and per-deployment salt $S \leftarrow \{0, 1\}^{256}$:

$$H_{\text{blinded},i} = \text{BLINDED-} \parallel \text{HMAC-SHA256}(S, P_i) \quad (4)$$

The blinded value retains the full 256-bit HMAC digest, preserving payload-aliasing resistance at the same scale as the underlying hash. It replaces raw payload strings in leaf construction while preserving global Merkle tree inclusion verification.

3.3 Merkle Trace Aggregation and Sparse Proof Compression

To attest N execution traces without transmitting full raw logs to every gate request, EviAssure aggregates normalized trace digests into a SHA-256 Merkle tree [17, 19]. Each trace record T_i maps to leaf digest H_i .

For $N = 1,000$ traces, transmitting raw trace logs requires 222.6 KB of storage, scaling to 220,705.03 KB for $N = 1,000,000$ traces. EviAssure provides sparse Merkle proof compression, transmitting the 32-byte root R_M and $O(\log N)$ inclusion proof paths π_i for audited steps.

Given proof path $\pi_i = \{(P_1, \text{pos}_1), \dots, (P_K, \text{pos}_K)\}$, inclusion is verified recursively by setting $C_0 = H_i$ (the committed leaf digest; leaf digests are already SHA-256 strings and are *not* re-hashed at the proof boundary, matching `verify_merkle_proof`) and evaluating:

$$C_j = \begin{cases} \text{SHA256}(C_{j-1} \parallel P_j), & \text{if pos}_j = \text{right} \\ \text{SHA256}(P_j \parallel C_{j-1}), & \text{if pos}_j = \text{left} \end{cases} \quad (5)$$

The proof is valid if and only if $C_K = R_M$ and the path length equals the committed tree depth $\lceil \log_2 N \rceil$, derived from the signed execution trace count; the length check prevents internal node digests from being verified as leaves—a substitution that would otherwise succeed without any hash collision—reducing transmission size to < 5 KB.

```
def build_merkle_tree(traces):
    leaves = [t.to_hash() for t in traces]
    current = leaves
    levels = [leaves]
    while len(current) > 1:
        next_level = []
        for i in range(0, len(current), 2):
            left = current[i]
            right = current[i+1] if i+1 < len(current)
            else current[i]
            next_level.append(hash_sha256(f"{left}:{right}"))
        current = next_level
        levels.append(current)
    return current[0], levels
```

Listing 1: Merkle Trace Aggregation Protocol.

3.4 SLSA v1.0 and in-toto Predicate Envelopes

To ensure seamless integration with modern cloud-native artifact registries and software supply chain attestors, EviAssure

formats each generated `EvidenceBundle` into an in-toto v0.2 Statement envelope compatible with SLSA Provenance v1.0 specifications [29,31]. Listing 2 illustrates the JSON structure exported by EviAssure.

The envelope schema binds the target release artifact (e.g., agent model weight package, code container, or configuration tarball) to the evidence attestation. The subject field specifies the artifact filename and its SHA-256 digest. The predicateType URI asserts SLSA Provenance v1.0 compliance. Within predicate.buildDefinition, the buildType asserts policy gate evaluation parameters, while externalParameters anchors the unique evidence_id, binding the external evidence pack directly to the SLSA build statement.

```
{
  "_type": "https://in-toto.io/Statement/v0.1",
  "subject": [
    {
      "name": "agentic_release_artifact.tar.gz",
      "digest": { "sha256": "03126da4143384d8..." }
    }
  ],
  "predicateType": "https://slsa.dev/provenance/v1",
  "predicate": {
    "buildDefinition": {
      "buildType": "https://usenix.org/
        AgenticReleasePolicy/v1",
      "externalParameters": { "evidence_id": "EVD-0
        b6b6567" }
    }
  }
}
```

Listing 2: SLSA Provenance v1.0 Statement envelope produced by EviAssure.

4 Fail-Closed Policy Verification Engine

Release authorization in EviAssure is governed by a zero-trust policy engine (`ReleasePolicyEngine`) that enforces mandatory fail-closed default rules. This section details the declarative policy schema, multi-factor evaluation protocol, key governance rules, and the system-level formal security analysis.

4.1 Declarative Policy Governance Rules

A release policy specification P defines strict quantitative safety thresholds and cryptographic verification conditions:

$$P = (P_{\min_pass}, P_{\max_drift}, P_{\min_sigs}, P_{\text{valid_window}}, \mathcal{K}_{\text{trusted}}, \mathcal{K}_{\text{rev}}, \mathcal{A}_{\text{KMS}}) \quad (6)$$

where:

- $P_{\min_pass}$: Minimum required unit and integration test pass rate (default: 100.0% (100/100)).
- $P_{\max_drift}$: Maximum allowable unresolved security drift findings (default: 0).

- $P_{\min_sigs}$: Required threshold number of valid signatures from authorized keys (default: $K = 1$).
- $P_{\text{valid_window}}$: Evidence timestamp freshness validity window $\Delta T_{\max} = 3,600$ s (1 hour) with a maximum future clock skew tolerance of $\delta_{\text{skew}} = 30$ s.
- $\mathcal{K}_{\text{trusted}}$: Trusted key registry pinning authorized signing key IDs to their Ed25519 public keys (`governance/trusted_keys.yaml`). Signature verification uses the pinned key, never a public key supplied inside a submitted bundle; unregistered key IDs are rejected. The registry follows the key-accountability patterns of key transparency [18] and delegation hierarchies [15]: verification authority rests on a small, versioned, offline-pinnable root set rather than on keys that travel with the evidence.
- $\mathcal{K}_{\text{rev}}$: Key Revocation List (CRL) tracking compromised or deprecated key identifiers.
- $\mathcal{A}_{\text{KMS}}$: Required Cloud KMS or HSM key ARN boundary pattern matching.

Listing 3 outlines the complete zero-trust verification algorithm enforced by `ReleasePolicyEngine`.

```
# Protocol 2: Fail-Closed Policy Gate Verification
# Input: Evidence E, Policy P, Revoked keys K_rev, Seen nonces N_seen
# Output: Decision D in {APPROVED, BLOCKED}
def evaluate_release_gate(E, P, K_rev, N_seen):
    if not E.signed or E.signature is None:
        return BLOCKED, ["POLICY_VIOLATION:_Unsigned_payload"]

    # 1. Signature & Key Governance Verification
    valid_sigs = 0
    for s_entry in E.signatures:
        if s_entry.key_id in K_rev or s_entry.key_id not in K_trusted:
            continue # revoked or unregistered key
        if s_entry.pub_key != K_trusted[s_entry.key_id]:
            continue # spoofed key_id under an attacker key
        if s_entry.kms_arn and not match_kms_arn(s_entry.kms_arn, P.kms_pattern):
            continue
        if verify_signature(s_entry):
            valid_sigs += 1

    if valid_sigs < P.min_signatures:
        return BLOCKED, ["POLICY_VIOLATION:_Insufficient_signatures"]

    # 2. Merkle Root Integrity Verification
    recalculated_root, _ = build_merkle_tree(E.traces)
    if recalculated_root != E.merkle_root:
        return BLOCKED, ["POLICY_VIOLATION:_Merkle_root_mismatch"]

    # 3. Quality & Security Drift Thresholds
    if E.test_pass_pct < P.min_pass_pct:
        return BLOCKED, ["POLICY_VIOLATION:_Sub-threshold_pass_rate"]
    if E.unresolved_drift > P.max_drift:
        return BLOCKED, ["POLICY_VIOLATION:_Unresolved_drift"]

    # 4. Nonce Replay & Freshness Windows
    if E.nonce in N_seen:
        return BLOCKED, ["POLICY_VIOLATION:_Replayed_nonce"]
    if not is_timestamp_fresh(E.timestamp, P.valid_window, max_skew=30):
```

```

    return BLOCKED, ["POLICY_VIOLATION:_Timestamp_
        expired/skewed"]

N_seen.add(E.nonce)
return APPROVED, []

```

Listing 3: Fail-Closed Policy Gate Verification Protocol.

4.2 Formal Security Analysis

The security analysis proceeds in two layers. Theorem 1 is the primitive-level argument: a sequence of games showing what no adversary can do to the cryptographic components, reducing to EUF-CMA and collision resistance. Theorem 2 is the system-level argument: it states what an APPROVED verdict *means* — the actual security objective of a release gate — and makes explicit the trust boundaries (evidence collector, key registry, nonce state) on which that meaning rests.

Theorem 1 (Fail-Closed Release Soundness). *Under the assumption that Ed25519 signatures are EUF-CMA secure and SHA-256 is collision-resistant, no probabilistic polynomial-time adversary $\mathcal{A}$ can induce `ReleasePolicyEngine` to output APPROVED for a modified trace history, forged signature, replayed nonce, or non-compliant evidence payload except with negligible probability $\text{negl}(\lambda)$.*

Proof. The proof structures the security argument using a formal sequence of games $\text{Game}_0 \rightarrow \text{Game}_1 \rightarrow \text{Game}_2 \rightarrow \text{Game}_3 \rightarrow \text{Game}_4$:

- **Game₀**: Standard execution of adversary $\mathcal{A}$ interacting with an honest evidence packager and policy gate verifier.
- **Game₁**: Identical to Game₀, except the verifier aborts if $\mathcal{A}$ produces a valid signature σ^* over a forged payload E^* under key PK without SK . The probability of $\mathcal{A}$ distinguishing Game₀ from Game₁ is bounded by the EUF-CMA security of Ed25519:

$$|\Pr[G_0] - \Pr[G_1]| \leq \text{Adv}_{\text{Ed25519}}^{\text{euf-cma}}(\mathcal{A}) \quad (7)$$

- **Game₂**: Identical to Game₁, except the verifier aborts if $\mathcal{A}$ produces a trace set $\mathcal{T}^* \neq \mathcal{T}$ such that $\text{MerkleRoot}(\mathcal{T}^*) = R_M$. The verifier additionally binds the claimed trace count to the recomputed tree and inclusion proofs to the committed depth, so internal-node substitution itself requires a collision. The difference between Game₁ and Game₂ is bounded by the collision resistance of SHA-256:

$$|\Pr[G_1] - \Pr[G_2]| \leq \text{Adv}_{\text{SHA256}}^{\text{coll}}(\mathcal{A}) \quad (8)$$

- **Game₃**: Identical to Game₂, except the verifier aborts if $\mathcal{A}$ submits a replayed evidence nonce $N^* \in N_{\text{seen}}$. Nonces are recorded in the state memory N_{seen} over the validity window, so the probability of within-window

reuse acceptance in Game₃ is identically zero; replays after expiry are rejected by the freshness check of Game₀'s policy evaluation.

- **Game₄**: Identical to Game₃, except the verifier aborts if $\mathcal{A}$ uses a revoked key $K^* \in \mathcal{K}_{\text{rev}}$ or an out-of-bounds KMS ARN. Rejection in Game₄ is deterministic.

Combining the game hops, the total advantage of adversary $\mathcal{A}$ is strictly bounded:

$$\begin{aligned} \text{Adv}_{\text{EviAssure}}^{\text{tamper}}(\mathcal{A}) &\leq \text{Adv}_{\text{Ed25519}}^{\text{euf-cma}}(\mathcal{A}) + \text{Adv}_{\text{SHA256}}^{\text{coll}}(\mathcal{A}) \\ &\leq \text{negl}(\lambda) \end{aligned} \quad (9)$$

This completes the proof. $\square$

Theorem 1 bounds an adversary's ability to *fake* evidence; it says nothing on its own about what a genuine, honestly-signed evidence pack entitles its holder to. That is the release-gate question, and it is Theorem 2.

Theorem 2 (Evidence-Bound Release Authorization Soundness). *Fix a policy version P and a trusted key registry $\mathcal{K}_{\text{trusted}}$. If `ReleasePolicyEngine` outputs APPROVED for evidence E , then, except with probability $\text{negl}(\lambda)$, all of the following hold:*

- Authenticity**: E carries a valid Ed25519 signature by a non-revoked key pinned in $\mathcal{K}_{\text{trusted}}$; attacker-generated keys, spoofed key IDs, and revoked keys cannot produce a valid signature.
- Evidence binding**: the recomputed Merkle root over the submitted trace set equals the signed root, the signed trace count equals the number of submitted traces, and every inclusion proof binds to the committed tree depth; the recorded evidence therefore corresponds exactly to what was submitted.
- Freshness**: the timestamp lies within the validity window ΔT_{max} and the skew bound δ_{skew} , and the nonce has not been processed within the window; the release is not a replay of an earlier approved bundle.
- Policy compliance**: all declared thresholds (test pass rate, unresolved drift, KMS ARN boundary, signature threshold K) satisfy policy version P ; an APPROVED verdict under a different policy version is not an APPROVED verdict for this release.

The guarantees hold conditionally on (a) the collector assumption of Definition 5 — the evidence collector observed and recorded every action of the release execution — which supplies the completeness half of the guarantee, and (b) the key-management assumption — registry keys are honestly generated and the HSM/KMS enforces the ARN boundary. The theorem asserts nothing about actions that never reached the collector, nor about the semantic correctness of recorded trace content; both are scoped in Section 2.3 and handled by the inspection layer (Section 6.9).

Proof. (i) Follows from Theorem 1, Game₁, plus the deterministic registry checks enforced before any signature verification (`policy.py`): unknown key IDs, bundle-supplied keys disagreeing with the registry, and revoked keys are rejected outright, so per-signature success is bounded by $\text{Adv}_{\text{Ed25519}}^{\text{euf-cma}}(\mathcal{A})$. (ii) Follows from Theorem 1, Game₂, plus the deterministic signed-count check $N = |\text{traces}|$; substitution requires either a hash collision or a broken signature. (iii) Freshness, skew, and nonce checks are deterministic (Game₃; Listing 3) and hold with probability 1 given correct gate state, with the disclosed caveat that replay protection requires sharing the nonce cache across gate replicas (Section 7.2). (iv) Threshold comparisons are deterministic over the signed payload and the pinned policy version. Composition yields the statement; the only probabilistic terms reduce to the primitive advantages of Theorem 1. $\square$

5 Real-World Case Study and Production CI/CD Integration

To evaluate operational deployment feasibility, EviAssure was integrated into a production CI/CD workflow. This section presents the GitHub Action architecture, browser UI attestation runner, and the 1,050-profile agent execution trace corpus.

EviAssure was deployed as a production GitHub Action ([.github/actions/evidence-release-gate/action.yml](https://github.com/evidence-release-gate/action)) and an in-toto v0.2 layout specification ([governance/intoto_layout.json](https://github.com/evidence-release-gate/governance/intoto_layout.json)). Evaluation tested two primary agentic workflow specimens:

1. **Backend Multi-Step Workflow Specimen** ([specimens/agent_runner.py](https://github.com/evidence-release-gate/specimens/agent_runner.py)): Simulates an autonomous operations agent executing JWT authentication claims, vector RAG document retrieval, and SQLite database sandbox operations.
2. **Browser UI Action Specimen** ([specimens/web_app_runner.py](https://github.com/evidence-release-gate/specimens/web_app_runner.py)): Simulates a web-browser autonomous agent executing page navigation, button clicks, DOM state hashing ($\text{SHA256}(\text{DOM}_{\text{serialized}})$), and screenshot render-reference digests ($\text{SHA256}(\text{render})$).

5.1 Production GitHub Action Architecture

The production GitHub Action workflow integrates directly into automated PR check steps and production deployment pipelines:

1. **Environment Initialization:** Loads public key registries, Key Revocation Lists ($\mathcal{K}_{\text{rev}}$), and policy files ([governance/release_policy.yaml](https://github.com/evidence-release-gate/governance/release_policy.yaml)).
2. **Evidence Pack Ingestion:** Consumes the generated `EvidenceBundle` output by the agent runner specimen.
3. **Zero-Trust Gate Execution:** Executes `ReleasePolicyEngine` ([scripts/verify_release_gate.py](https://github.com/evidence-release-gate/scripts/verify_release_gate.py)).

If verification fails, the action logs violation messages, outputs structured diagnostic JSON, and exits with code 1 (BLOCKED).

4. **Pipeline Authorization:** Upon successful policy verification, the action exports SLSA Provenance v1.0 statements and authorizes downstream deployment jobs.

5.2 Expanded Agent Execution Trace Corpus ($N = 1,050$)

A 1,050-profile benchmark corpus ([corpus/agent_trace_corpus.json](https://github.com/evidence-release-gate/corpus/agent_trace_corpus.json)) was constructed across five agent architectures (CodeSynthesisAgent, MultiStepRAGAgent, DatabaseAdminAgent, FinancialAPIIntegratorAgent, AutonomousWebScraperAgent): 1,000 clean profiles and 50 anomalous profiles spanning 10 planted anomaly types (indirect prompt injection, privilege escalation, unauthorized shell spawning, database drops, secret exfiltration, memory trace tampering, SSRF, vector store poisoning, un-sandboxed process creation, JSON key malleability) at five profiles per type. Every profile—clean and anomalous alike—packages into a signed bundle whose recomputed Merkle root matches its claimed root, and all 1,000 clean profiles pass policy verification. The planted anomalies are semantic (visible in trace status fields) rather than cryptographic: because anomalous profiles remain well-formed signed bundles, they surface through per-leaf forensic inspection (Section 5.3) instead of the cryptographic gate, whose blocking behavior is measured over the 12 tamper vectors (Section 6.6) and whose two-layer behavior over the full corpus is measured in Section 6.9.

5.3 CLI Forensic Inspection & Audit Tool

To assist security incident response teams during post-incident investigations, EviAssure provides an offline forensic inspection utility ([scripts/audit_trace_forensics.py](https://github.com/evidence-release-gate/scripts/audit_trace_forensics.py)). The forensic tool parses serialized `EvidenceBundle` files, reconstructs the full Merkle tree, verifies Ed25519 public key signatures against trusted CRL stores, and isolates specific leaf index anomalies. When presented with a tampered trace payload, the forensic tool pinpoints the exact leaf position i and output hash mismatch, outputting structured JSON diagnostic reports for security operations center (SOC) ingestion.

6 Empirical Evaluation

Performance evaluation was conducted across four dimensions: Merkle tree scaling latency, multi-core verifier throughput, adversarial tamper resilience across 12 attack vectors, and a corpus-scale two-layer evaluation of cryptographic integrity versus per-leaf semantic inspection (Section 6.9).

All timed measurements are wall-clock timings recorded by [scripts/run_release_benchmark.py](https://github.com/evidence-release-gate/scripts/run_release_benchmark.py) into [results/benchmark_summary.json](https://github.com/evidence-release-gate/results/benchmark_summary.json) on an Apple M-series platform

with 14 logical cores (10 performance, 4 efficiency) running Python 3.14.x. Each scaling point, throughput configuration, proof cost, blinding cost, and hashing figure is the mean over 5 runs, with the standard deviation recorded alongside in the artifact (*_std fields); per-worker throughput aggregates 1,000 policy evaluations per run. Tamper vectors and gate verdicts are deterministic checks evaluated once each, so run-to-run dispersion there is zero by construction [23, 34].

6.1 Merkle Scaling up to $N = 1,000,000$ Traces

Packaging latency and Merkle tree build time were evaluated as trace size increased from $N = 10$ up to $N = 1,000,000$ over 5 runs per point; Merkle construction is timed on a single build per run and packaging latency excludes it, so the two numbers are measured on one build rather than two redundant ones. As shown in Figure 1, Merkle tree construction scales efficiently to 1,000,000 traces in 419.21 ms (mean; $\sigma = 27.21$ ms across runs), and to 39.3 ms for $N = 100,000$, a packaging overhead of 0.0002% at $N = 100,000$ relative to trace execution duration under the benchmark’s synthetic 1.5 ms per-step execution time (the denominator is a benchmark parameter, so the ratio is an order-of-magnitude illustration, not a production cost model).

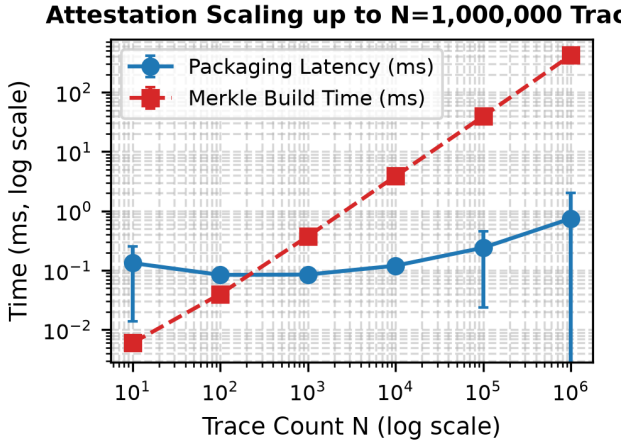


Figure 1: Attestation packaging latency and Merkle tree build time as trace count N scales up to 1,000,000.

6.2 Multi-Core Parallel Verifier Throughput

Verifier throughput was evaluated using a Python `ProcessPoolExecutor` across 1, 2, 4, 8, and 16 worker cores over 1,000 evaluation requests per run and 5 runs per configuration; each evaluation is a distinct fresh-nonce bundle against a warm policy engine (evidence creation and replay-state checks are measured separately, so the timed region is steady-state verification). Figure 2 shows

throughput rising from 3,802 ops/s on a single worker to a peak of $7,317 \pm 79$ operations/second (mean $\pm$ std) at 4 workers (a $1.9\times$ speedup), then declining as additional workers contend for memory bandwidth and, at 16 workers, oversubscribe the machine’s 14 physical cores.

6.3 Sparse Merkle Inclusion Proof Compression & Audit Latency

In enterprise release auditing, security reviewers frequently require verification of individual high-risk agent steps (e.g., database schema alterations or financial API calls) without re-processing full trace logs (220,705 KB at $N = 1,000,000$). EviAssure’s $O(\log N)$ sparse Merkle inclusion proof generator extracts compact verification paths for target leaf indices. For $N = 1,000,000$ traces (tree depth 21), proof path generation requires 0.003 ms (mean over 5 runs), yielding a 1.93 KB proof payload containing 20 SHA-256 node digests. Inclusion verification against root R_M requires 0.009 ms, a bandwidth reduction of roughly 99.999% compared to full log transmission (1.93 KB vs. 220,705 KB).

6.4 Salted Trace Parameter Blinding Performance

To evaluate the runtime overhead of privacy-preserving parameter blinding (`blind_payload`), 10,000 execution traces containing sensitive prompt texts and PII strings were processed using keyed HMAC-SHA256 hashes with a 256-bit salt over 5 runs. Parameter blinding added 0.0014 ms of latency per trace record (14.5 ms total for $N = 10,000$), demonstrating that privacy protection imposes negligible performance overhead.

6.5 Browser UI DOM State & Screenshot Render Attestation

Browser agent UI action attestation was evaluated across 1,000 automated web interaction steps using `WebBrowserAgentSpecimen`. SHA-256 hashing of the serialized DOM state averaged 0.0014 ms per step over a realistic multi-node HTML fragment (the same serialization the specimen attests), and the screenshot render-reference digest ($\text{SHA256}(\text{render})$) averaged 0.0004 ms per step. When an adversarial DOM mutation (e.g., injecting an unauthorized modal dialog) is introduced, the altered page state changes H_{DOM} and thus the committed leaf digest, and re-signing over the altered trace yields a recomputed Merkle root that no longer matches the signed root, triggering fail-closed gate rejection (BLOCKED); a mutation applied *after* signing leaves the signed root unchanged while the bundle’s own trace data disagrees with it, which the same root-recompute check rejects.

6.6 12-Vector Adversarial Tamper Resilience

Table 1 presents detailed security benchmark results across all 12 adversarial tamper vectors, mapping each attack vector to its failure category, expected action, and observed gate verdict.

EviAssure achieved a 100.0% fail-closed block rate (12/12 vectors successfully blocked). The mitigation mechanics operate as follows:

- **V1 (Unsigned Payload):** Blocked by strict enforcement of `signed == true`.
- **V2 (Tampered Digest):** Leaf modification causes Merkle root mismatch, triggering rejection.
- **V3 (Forged Signature):** Bundles signed by attacker-generated keypairs are blocked by trusted-key-registry enforcement: unregistered key IDs are rejected, and spoofing a registered key ID fails the pinned-key comparison.
- **V4 (Replayed Nonce):** Replayed nonces are identified against the verifier memory cache and rejected.
- **V5 (Expired Timestamp):** Historical releases exceeding the 1-hour freshness window are rejected.
- **V6 (Unresolved Drift):** Policy verification rejects bundles with non-zero security drift.
- **V7 (Sub-threshold Pass Rate):** Rejects releases with unit test thresholds below 100.0% (100/100).
- **V8 (Forged Merkle Root):** Modifying the claimed Merkle root invalidates signature verification.
- **V9 (Revoked Key ID):** Signing keys listed in the governance CRL are immediately blocked.
- **V10 (JSON Malleability):** A wire document that repeats a signed key (e.g., `test_pass_pct` appearing twice with different values) is parser-ambiguous—last-wins versus first-wins parsers disagree on the value. The ingestion boundary detects the duplicate keys in the raw wire text and rejects the document as non-canonical before any policy or signature logic runs; the canonical JSON serialization of the policy-relevant payload is what the signature covers, so no parser disagreement can change the verdict.
- **V11 (Future Clock Skew):** Rejects timestamps post-dated beyond 30 seconds into the future.
- **V12 (Truncated Merkle Path):** Removing traces changes the recomputed Merkle root, triggering a mismatch rejection.

6.7 Per-Check Ablation

To isolate the marginal contribution of each enforcement, the gate was re-evaluated over all 12 vectors with exactly one check disabled per run (condition overrides on the same engine and registry; matrix recorded in the artifact’s `ablation` block). Disabling the signature check releases its three targeted vectors (V1 unsigned, V3 attacker key, V9 revoked key);

disabling the Merkle re-derivation releases the digest and root forgeries (V2, V8); the quality, drift, freshness, and replay checks each release exactly their targeted vectors (V7; V6; V5 and V11; V4). Two structural results stand out. First, removing the Merkle check does *not* release the truncation vector V12: the signed trace-count binding independently catches it, so the tree geometry and the root are protected by two disjoint checks. Second, removing the trusted-key registry does not release any vector — instead the gate fail-closes, rejecting *all* Ed25519 evidence because signatures can no longer be authenticated; the registry is the trust anchor whose removal disables authorization entirely rather than weakening it selectively. Every check is necessary (its removal releases at least one vector it uniquely targets, or collapses the gate), and no single check is sufficient.

6.8 Comparative Deployment Gate Analysis

Figure 3 compares EviAssure against the three deployment gate types organizations compose today. Rather than presenting a single “OPA/Sigstore” baseline—those technologies address different concerns, and merging them without justification would obscure what is compared—we configure each gate to enforce the subset of EviAssure’s semantics its technology natively offers, and report baselines separately (per-baseline execution record: `baseline_execution`):

1. **Standard CI exit-code gate** (*modeled*, exit-code semantics): passes iff `test_pass_pct > 0` and no hard-failure flag is set. It has no view of signatures, trace integrity, freshness, or governance, so it blocks only the quality vectors it can see: **0/12**.
2. **OPA schema-policy gate** (*executed*, OPA 1.19.0): the same quality and drift thresholds EviAssure enforces (pass rate $\geq 100\%$, zero unresolved drift), expressed as a Rego policy over the evidence JSON and evaluated by `opa eval (governance/baseline_opa.rego)`. OPA is a policy *engine*; it takes the submitted payload as unauthenticated input, so an attacker who tampers with the declared field values defeats it. It blocks exactly the pure policy-mismatch vectors: **3/12** (V6, V7, V10).
3. **Sigstore/Cosign-style artifact-signature gate** (*modeled*, artifact-signature presence): requires a signature over the artifact digest—the assurance class Cosign provides for container images: proof that *someone* signed the artifact, with no execution-trace semantics and no quality-field coupling. It blocks only unsigned payloads (**1/12**, V1) and is blind to sub-threshold pass rates, tampered leaves, forged roots, replay, revocation, and skew.

EviAssure blocks **12/12**. The comparison’s claim is deliberately narrow: no single existing gate type spans the signed-trace + replay + governance axis, and the 12 vectors exercise exactly that axis. Modeled baselines are disclosed as such—they capture the guarantee classes those tools offer over an

evidence payload (exit-code semantics and artifact-signature presence), not executions of the CI runner or `cosign`—while the OPA baseline is a real execution over the same tampered payloads. A composed “OPA+Sigstore” deployment still lacks trace attestation, nonce replay protection, and signed trace-count binding, which is precisely the gap EviAssure closes.

6.9 Corpus-Scale Two-Layer Evaluation

The 12-vector suite exercises the cryptographic gate; it cannot exercise the semantic axis, because a well-formed signed bundle recording unauthorized behavior does not violate any cryptographic check. The 1,050-profile corpus (Section 5.2) is the complementary evaluation: every profile is packaged into a signed bundle and evaluated through both layers independently (`scripts/run_corpus_eval.py` → `results/corpus_evaluation.json`). Table 2 reports the results.

Layer 1 confirms integrity at corpus scale: all 1,050 bundles, clean and anomalous alike, are Merkle-consistent with fully valid depth-bound inclusion proofs, by construction of the signed count binding (Section 4.2). Layer 2 confirms the gate’s behavior: all 1,000 clean profiles are APPROVED; the 50 anomalous profiles are *also* APPROVED, and this is reported rather than hidden—semantic misbehavior that is cryptographically well-formed lies outside the gate’s scope (Section 7.2). Layer 3 is the complement: per-leaf inspection flags every anomalous profile (50/50, each carrying at least one trace whose status is not SUCCESS or whose action is on the forensic sensitive-action list) with zero false positives over the 1,000 clean profiles, across 10 planted anomaly classes (prompt injection, privilege escalation, unauthorized shell spawning, database drops, secret exfiltration, memory trace tampering, SSRF, vector-store poisoning, un-sandboxed process creation, JSON key malleability). The layers are a division of labor, not alternatives: the gate denies what can be proven false; inspection surfaces what should not have happened even though it is provably genuine.

6.10 Discussion and Operational Deployment Trade-offs

Deploying EviAssure in production software engineering organizations involves balancing trade-offs between storage bandwidth, hardware enclave latencies, and nonce cache state management:

1. **Storage vs. Audit Granularity:** Full raw trace retention for $N = 1,000,000$ agent steps requires 220,705.03 KB per release. EviAssure’s $O(\log N)$ sparse Merkle proof generator transmits 1.93 KB proof paths ($\sim 99.999\%$ storage reduction), enabling organizations to store lightweight inclusion proofs on-chain or in im-

mutable ledger databases while archiving raw payloads to cold storage.

2. **Hardware Security Enclave Overhead:** Offloading Ed25519 signature generation to Cloud KMS or HSM endpoints introduces an additional network round-trip ($\approx 12\text{--}25$ ms per signature call). To prevent CI pipeline bottlenecks, EviAssure batches evidence root digests into multi-signature calls or employs local worker process pools.
3. **Stateful Nonce Cache Memory Footprint:** Replay protection requires the verifier engine to maintain a memory cache of active nonces N_{seen} over the validity window $\Delta T_{\text{max}} = 3,600$ s. At a rate of 1,000 releases per second, storing 3.6×10^6 UUID nonces requires approximately 115 MB of RAM, representing a modest memory footprint for enterprise gateway nodes.

7 Related Work

Table 3 compares EviAssure against existing software supply chain and policy gate frameworks across six security dimensions.

7.1 Capability-Class to Vector Mapping

Table 4 traces each adversary capability class of Section 2.3 to the tamper vectors that exercise it and the enforcement that stops it. V6 and V7 are policy-compliance vectors rather than capabilities of $\mathcal{A}$: they release a correctly signed bundle whose quality gates fail.

7.2 Limitations and Threats to Validity

Measurement methodology. All timed figures are means over 5 runs with standard deviations recorded in the artifact; the repeats are not independent draws across machines or days, so the reported σ captures within-run noise only, and run-to-run variation on the order of a few percent across sessions should be assumed. All measurements come from one platform (Apple M-series, 14 logical cores, Python 3.14.x, recorded in the artifact’s `platform` block); cross-platform behavior was not measured. **Key management.** The artifact signs with a static demo key pinned in `governance/trusted_keys.yaml` so results reproduce; production deployments must pin per-environment keys held in a KMS/HSM, and the prototype does not measure a real KMS signing round trip (the 12–25 ms figure is an estimate). **Baseline realism.** The OPA baseline is executed (OPA 1.19.0, Rego policy in the artifact); the CI and Sigstore/Cosign baselines are modeled abstractions, as disclosed in Section 6.8. **Completeness boundary.** The security theorems hold for actions the evidence collector observed; an adversary that controls the recording path before collection can omit actions,

and no cryptographic check can expose the omission (Sections 2.3 and 2). **Scope of blocking.** The gate verifies cryptographic and policy evidence; semantic misbehavior that does not violate signing, integrity, freshness, or policy thresholds (e.g., a well-signed bundle recording unauthorized actions) is surfaced by per-leaf forensic inspection rather than blocked at the gate — content-level policy over trace semantics is future work, and the corpus evaluation (Section 6.9) reports the inspection layer’s 50/50 recall and 0/1000 false-positive rate on the planted anomalies. Replay state (N_{seen}) is process-local: a gate restart resets it within the validity window, so deployments must share nonce state across replicas to preserve replay protection. **Construct validity.** The 12 vectors were authored by the authors to exercise each enforcement check; a 100% block rate (12/12 vectors) demonstrates the checks fire, not resilience to novel attacks. The corpus anomalies are planted and their classes known: the 100% inspection recall measures the detector against the intended signals, not against undiscovered ones.

7.3 Software Supply Chain Security Frameworks

Software supply chain security frameworks such as in-toto [31], SLSA [29], and Sigstore/Cosign [21] provide cryptographic provenance for static build pipelines and container image digests. In-toto records step-level layout attestations, and its attestation framework ships vetted predicates including runtime traces, test results, and SCAI evidence; SLSA builds four graduated levels of build-environment isolation on top. These systems attest *what happened* in a build, and any release pipeline should leverage them for the artifact half of the story. What they do not provide is an authorization layer for agent releases: no freshness window or replay cache is integral to step attestations (a verifier may enforce or ignore them), no release decision is bound to the attested evidence under a pinned policy version, and non-deterministic multi-step LLM agent traces, browser DOM mutations, and salted blinding have no native representation. EviAssure supplies that missing half while remaining envelope-compatible: its `EvidenceBundle` is an in-toto Statement that can carry SLSA provenance, an `agent-decision`, or any other predicate alongside the Merkle-attested trace pack (Section 7.7).

7.4 Policy Gate Engines & Admission Controllers

General-purpose policy engines such as Open Policy Agent (OPA) [27] and Kyverno [30] evaluate declarative Rego or YAML policy rules over JSON admission requests. While effective for Kubernetes cluster governance, OPA and Kyverno do not compute Merkle inclusion roots over action histories, nor do they maintain stateful nonce replay caches or key

revocation list (CRL) validation out-of-the-box. As demonstrated in our comparative empirical benchmarks, evaluating unauthenticated admission payloads without trace attestation leaves gates vulnerable to trace tampering and replay attacks.

7.5 Dynamic Capability & Workload Identity Tokens

Authorization frameworks such as Macaroons [5] and SPIFFE/SPIRE [11] provide cryptographic capability tokens with contextual caveats and short-lived X.509 workload identities. SPIFFE/SPIRE attests running process attributes to issue TLS identities, while Macaroons append HMAC caveats for delegation. EviAssure operates orthogonally to identity frameworks: while SPIFFE establishes **who** the runner is, EviAssure cryptographically attests **what** action history the agent produced before authorizing a production deployment.

7.6 Runtime LLM Guardrails vs. Release Assurance

Runtime LLM guardrail platforms (e.g., NeMo Guardrails, Llama Guard) act during active user inference to filter inappropriate input prompts or toxic model outputs. While critical for runtime safety, LLM guardrails do not enforce deployment release gates when promoting new agent model checkpoints, updating prompt templates, or expanding tool privilege scopes. EviAssure complements runtime guardrail systems by establishing a cryptographically attested, fail-closed release gate for pre-deployment validation [29].

7.7 AI-Agent Provenance and Auditability Standards (2026)

The research landscape moved in 2026; three developments are closest to EviAssure’s territory, and all three are *record-format* efforts rather than authorization layers:

- **in-toto agent-decision/v0.1 predicate [2]:** a May 2026 RFC proposes registering an in-toto predicate for AI-agent policy decisions—agent identity, principal, evaluated policies, allow/deny outcomes, hashed tool arguments, timestamps, and trace correlation, signed with Ed25519 DSSE. It standardizes the *shape* of a decision record; it does not enforce a release decision, and freshness, replay, and revocation remain verifier-side choices.
- **Agent Provenance Attestation Standard (APAS) [1]:** a draft standard for cryptographically verifiable provenance chains across autonomous agent pipelines (work decomposition → execution → code delivery), built on in-toto Statement envelopes with a `dispatch` predicate. APAS grades itself: at lower levels provenance is “self-attested — an attacker who compromises the orchestrator can forge records,” and external witnessing is deferred

to its top level. EviAssure’s layout differs on the axis APAS itself flags: signing (KMS/HSM + pinned registry) is separated from recording, and the release decision is reproducible by an independent verifier without trusting the orchestrator’s attestations alone.

- **Agent Auditability Standard (AAS-1)** [13]: a May 2026 draft defining an evidentiary record format for agent activity—five record classes, twelve audit assertions, JCS canonicalization, and per-issuer hash chains. Its hash chains let a verifier holding all records detect omission *between* consecutive records of one issuer; EviAssure makes the analogous binding structural rather than optional—the signed trace count ties the Merkle root to exactly the submitted action set—and additionally enforces the authorization semantics (freshness, replay, revocation, thresholds) that audit standards deliberately leave to the engagement.

Against all three, EviAssure’s contribution is the layer they omit: *evidence-bound, fail-closed release authorization*. Their records ride inside EviAssure’s in-toto-compatible envelope without modification, and EviAssure’s verification approach (pinned keys, signed counts, gate-stateful replay protection) applies to any predicate that fits a Statement. Table 3 situates these standards alongside the earlier frameworks.

8 Ethics and Dual-Use Analysis

This research addresses release assurance infrastructure for autonomous AI agents. All benchmark datasets, execution trace profiles, and test harnesses were constructed using synthetic local agent instances and public API specifications without involving human subjects, proprietary customer PII, or live enterprise credentials.

8.1 Dual-Use Considerations

Cryptographic execution attestation mechanisms, if repurposed maliciously, could theoretically be employed to construct invasive employee monitoring frameworks or enforce rigid software lock-in. EviAssure mitigates these dual-use concerns by enforcing decentralized multi-signature threshold governance, open policy schemas ([governance/release_policy.yaml](#)), and client-side salted HMAC parameter blinding (`blind_payload`). Blinding ensures that operational safety verification remains decoupled from intrusive payload inspection.

8.2 Data Privacy & Parameter Blinding

Autonomous software agents frequently process confidential prompt texts, customer financial details, or proprietary source code snippets. To prevent public release audit logs from exposing sensitive data, EviAssure keeps raw payloads out of

evidence bundles entirely—traces carry digests only—and the blinding protocol additionally domain-separates those digests with a per-deployment salted HMAC (H_{blinded}), so the same recorded action is not linkable across environments. This preserves full Merkle tree inclusion auditability without storing proprietary inputs in the audit log.

8.3 Responsible Disclosure & Safety Impact

By enforcing a strict zero-trust fail-closed release gate, EviAssure provides immediate security benefits for organizations deploying autonomous agents into production. It prevents compromised runner environments or indirect prompt injection vulnerabilities from bypassing deployment safety gates, thereby reducing the systemic risk of unauthorized AI agent deployments in critical infrastructure.

9 Conclusion

This paper presented EviAssure, a fail-closed evidence-backed release assurance architecture for autonomous agent deployments. By coupling SHA-256 Merkle trace attestation with Ed25519 digital signatures, signed trace-count binding, and zero-trust policy verification, EviAssure replaces unauthenticated release signals with evidence-bound authorization: an APPROVED verdict is a deterministic, auditable function of attested evidence under a pinned policy version. A system-level Release Authorization Soundness theorem states what that verdict means and the trust boundaries it rests on—most importantly the explicit distinction between evidence integrity, which the cryptographic layer guarantees, and trace completeness, which holds at the evidence collector’s observation boundary. The empirical evaluation confirms a 100.0% (12/12) block rate across 12 tamper vectors (versus 0/12 for CI exit-code gates, 3/12 for an executed OPA schema gate, and 3/12 for a Sigstore-style signature gate), and a corpus-scale two-layer evaluation in which 1,050/1,050 profiles attest with valid inclusion proofs, 1,000/1,000 clean profiles pass the gate, and 50/50 planted anomalies are surfaced by per-leaf inspection with zero false positives. Positioned against the 2026 agent-provenance and auditability standards, EviAssure supplies the authorization layer those record formats omit, and remains envelope-compatible so the two compose.

References

- [1] Agentic Research. Agent provenance attestation standard (apas). Draft v0.2.1, reference implementation “rosary”/signet, 2026. Draft standard for cryptographically verifiable provenance chains across autonomous AI-agent pipelines, from work decomposition through agent execution to code delivery; uses in-toto Statement envelopes and DSSE signing.

- [2] Auxidus Technologies. Rfc: agent-decision/v0.1 predicate for ai agent policy decisions. in-toto/attestation issue #554, in-toto Attestation Framework (CNCF), 2026. Open RFC proposing an in-toto attachment predicate for AI agent authorization decisions: agent identity, principal, policy evaluations, allow/deny outcomes, hashed tool arguments, timestamps, and trace correlation.
- [3] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In *ACM Conference on Computer and Communications Security (ACM CCS)*, pages 62–73, 1993.
- [4] Daniel J Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012.
- [5] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrabie, and Mark Lentzner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *NDSS Symposium*, 2014.
- [6] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the weil pairing. In *ASIACRYPT*, pages 514–532, 2001.
- [7] Justin Cappos, Justin Samuel, Scott Baker, and John H Hartman. A look in the mirror: Attacks on package managers. In *ACM Conference on Computer and Communications Security (ACM CCS)*, pages 565–574, 2008.
- [8] Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Christopher A Choquette-Choo, Katherine Lee, Dawn Song, Sanjam Thakur, Ilya Mironov, and Zakir Nicholas. Poisoning web-scale training datasets is practical. In *IEEE Symposium on Security and Privacy (IEEE S&P)*, 2024.
- [9] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. Towards measuring supply chain attacks on package managers for interpreted languages. In *NDSS Symposium*, 2021.
- [10] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90, 2023.
- [11] CNCF Identity Working Group. Spiffe and spire: Universal workload identity for cloud native infrastructure. In *Cloud Native Computing Foundation (CNCF) Technical Report*, 2020.
- [12] in-toto Attestation Framework contributors (CNCF). Runtime trace predicate, in-toto attestation framework. Predicate specification, <https://github.com/in-toto/attestation/blob/main/spec/predicates/runtime-trace.md>, 2024.
- [13] Kadikoy Limited. Agent auditability standard (aas-1), working paper v0.1. Draft for public comment, companion to the AIS-1 agent identity standard, 2026. Open standard for audit-grade evidentiary records of autonomous agent activity: five record classes (Action, Batch, Continuous, Determination, Engagement), twelve audit assertions, JCS canonicalization, SHA-256 hashing, and per-issuer hash chains for omission detection.
- [14] Hugo Krawczyk, Mihir Bellare, and Ran Canetti. Hmac: Keyed-hashing for message authentication. In *RFC 2104, IETF*, 1997.
- [15] Trishank Karthik Kuppusamy, Santiago Torres-Arias, Vladimir Diaz, and Justin Cappos. Diplomat: Using delegations to protect community repositories. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI 16)*, 2016.
- [16] Piergiorgio Ladisa, Henrik Plate, Matías Martínez, and Olivier Barais. Sok: Taxonomy of attacks on open-source software supply chains. In *IEEE Symposium on Security and Privacy (IEEE S&P)*, pages 1509–1526, 2023.
- [17] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. In *RFC 6962, IETF*, 2014.
- [18] Marcela S Melara, Aaron Blankstein, Joseph Bonneau, Edward W Felten, and Michael J Freedman. Coniks: Bringing key transparency to end users. In *USENIX Security Symposium (USENIX Security 15)*, pages 383–398, 2015.
- [19] Ralph C Merkle. A certified digital signature. *Advances in Cryptology—CRYPTO’89 Proceedings*, pages 218–238, 1989.
- [20] Marina Moore, Trishank Karthik Kuppusamy, and Justin Cappos. Artemis: Defanging software supply chain attacks in multi-repository update systems. In *Annual Computer Security Applications Conference (ACSAC)*, pages 1002–1017, 2023.
- [21] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. Sigstore: Software signing for everybody. In *ACM Conference on Computer and Communications Security (ACM CCS), Demo Track*, 2022.

- [22] Chinenye Okafor, Taylor R Schorlemmer, Santiago Torres-Arias, and James C Davis. Sok: Analysis of software supply chain security by establishing secure design properties. In *ACM Conference on Computer and Communications Security (ACM CCS)*, pages 1811–1825, 2022.
- [23] Nicolas Papernot, Fartash Faghri, Nicholas Carlini, Ian Goodfellow, Reuben Feinman, Alexey Kurakin, Cihang Xie, Yash Sharma, Tom Brown, Aurko Roy, et al. Technical report on the cleverhans v1.0.0 adversarial examples library. In *Technical Report*, 2016.
- [24] Justin Samuel, Nick Mathewson, Justin Cappos, and Roger Dingledine. Survivable key compromise in software update systems. In *ACM Conference on Computer and Communications Security (ACM CCS)*, pages 177–190, 2010.
- [25] Roei Schuster, Congzheng Song, Eran Tromer, and Vitaly Shmatikov. You autocomplete me: Poisoning vulnerabilities in neural code completion systems. In *USENIX Security Symposium (USENIX Security 21)*, pages 1559–1575, 2021.
- [26] Adam Shostack. *Threat Modeling: Designing for Security*. John Wiley & Sons, Indianapolis, IN, 2014.
- [27] Torin Stichbury and Timmy Sandeep. Open policy agent (opa): Declarative fine-grained policy engine for cloud native environments. In *Cloud Native Computing Foundation (CNCF)*, 2021.
- [28] Ankur Taly, Úlfar Erlingsson, John C Mitchell, Mark S Miller, and Jasvir Nagra. Automated analysis of security-critical javascript apis. In *IEEE Symposium on Security and Privacy (IEEE S&P)*, 2011.
- [29] Google Open Source Security Team. Supply-chain levels for software artifacts (slsa) framework specification. In *Linux Foundation OpenSSF Technical Report*, 2023.
- [30] Nirmata Engineering Team. Kyverno: Kubernetes native policy management engine. In *Cloud Native Computing Foundation (CNCF)*, 2022.
- [31] Santiago Torres-Arias, Anish Kathpal, Hiten Gupta, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *USENIX Security Symposium (USENIX Security 19)*, pages 1393–1410, 2019.
- [32] John Yang, Carlos E Jimenez, Alexander Wettig, Tatiana Likhomanenko, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [33] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [34] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhonghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph E Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023.

A Open Science Appendix

The reproducibility materials consist of the complete Evi-Assure implementation (including cryptographic attestation, privacy blinding, and policy verification modules), the 1,050-profile agent execution trace corpus, the 12-vector tamper evaluation harness, production GitHub Action workflows, benchmark scripts, and the paper figure generator.

A.1 Anonymous Review Artifact & Verification

To comply with USENIX Security ’27 Open Science guidelines while preserving double-blind review, an anonymous review package `eviassure_usenix27_artifact.zip` (size: 168.1 KB) has been prepared using [scripts/prepare_anonymous_artifact.py](#). The archive carries SHA-256 digest:

7e2cc66eaf3a352d...

A.2 Reproduction Workflow

To execute full reproduction of paper figures, benchmark results, and unit tests:

1. **Environment Setup:** Install dependencies via `pip install -e .` (Python ≥ 3.10).
2. **Unit & Integration Test Suite:** Run `pytest tests/ -v` to execute all 42 unit, integration, property-based (crypto soundness), browser attestation, trusted-key-registry, tamper resilience, corpus-evaluation, adversarial-regression (2026-08 review findings), and docs-consistency tests.
3. **Attestation Scaling & Throughput Microbenchmarks:** Run `python3 scripts/run_release_benchmark.py -repeats 5` to generate [results/benchmark_summary.json](#).
4. **Adversarial Tamper Evaluation:** Run `python3 scripts/run_comparative_eval.py` to execute the 12-vector attack harness and produce [results/comparative_evaluation.json](#).

5. **Corpus Two-Layer Evaluation:** Run `python3 scripts/run_corpus_eval.py` to evaluate all 1,050 corpus profiles through the cryptographic, policy, and per-leaf inspection layers and produce [results/corpus_evaluation.json](#).
6. **Figure Generation & Manuscript PDF Rebuilding:** Run `python3 scripts/generate_paper_pdf.py` to regenerate paper plots and recompile [docs/usenix_paper_manuscript.pdf](#).

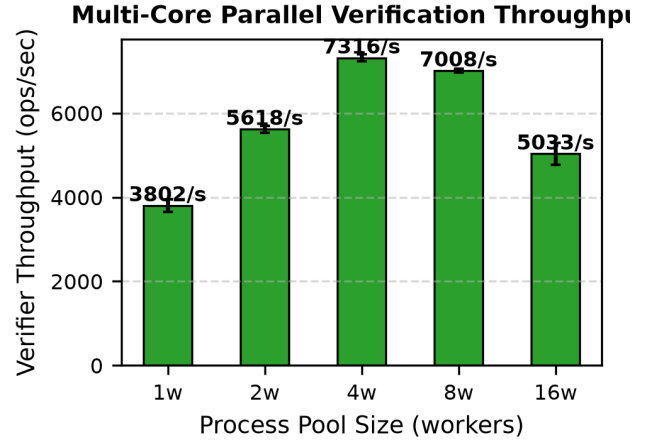


Figure 2: Multi-core parallel policy gate verification throughput across 1, 2, 4, 8, and 16 worker cores.

Table 1: Adversarial Release Tamper Vector Benchmark Results.

ID	Attack Vector	Cat.	Result	Status
V1	Unsigned Payload	Auth	Blocked	PASS
V2	Tampered Trace Digest	Integrity	Blocked	PASS
V3	Forged Signature	Auth	Blocked	PASS
V4	Replayed Nonce	Replay	Blocked	PASS
V5	Expired Timestamp	Fresh	Blocked	PASS
V6	Unresolved Drift	Policy	Blocked	PASS
V7	Sub-threshold Pass Rate	Quality	Blocked	PASS
V8	Forged Merkle Root	Integrity	Blocked	PASS
V9	Revoked Key ID	Key Gov	Blocked	PASS
V10	JSON Malleability	Serial	Blocked	PASS
V11	Future Clock Skew	Fresh	Blocked	PASS
V12	Truncated Merkle Path	Integrity	Blocked	PASS

Adversarial Release Tamper Detection (12 Vec)

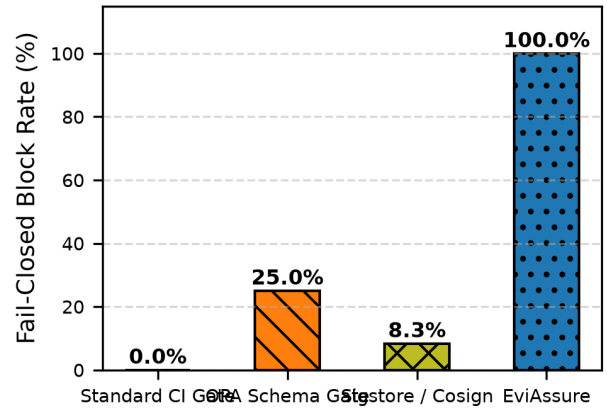


Figure 3: Comparative release tamper detection block rates across 12 attack vectors.

Table 2: Corpus-scale two-layer evaluation over 1,050 agent trace profiles.

Layer	Check	Result	Rate
L1 Integrity	Merkle root consistency	1050/1050	100.0%
L1 Integrity	Inclusion proofs valid	1050/1050	100.0%
L2 Policy gate	Clean profiles APPROVED	1000/1000	100.0%
L2 Policy gate	Anomalous APPROVED (dis-closed)	50/50	100.0%
L3 Inspection	Anomalous profiles flagged	50/50	100.0%
L3 Inspection	Clean falsely flagged	0/1000	0.0%
Distinct anomaly classes		10	

Table 3: Comparative analysis of EviAssure against existing software supply chain and policy gate frameworks.

System	Trace Attest.	Asym. Signing	SLSA v1.0	Fail-Closed	Replay Prot.	Agent-Spec.
SLSA Framework [29]	Baseline	Yes	Yes	Manual	Partial	No
in-toto [31]	Step-level	Yes	Partial	Manual	No	No
Sigstore / Cosign [21]	Artifact-level	Yes	Envelope	No	Partial	No
Open Policy Agent (OPA) [27]	No	Optional	No	Optional	Manual	No
Kyverno Policy Engine [30]	No	Cosign	No	Enforced	No	No
Macaroons Authorization [5]	Dynamic	HMAC	No	Enforced	Token	No
SPIFFE / SPIRE Workload ID [11]	Identity	X.509	No	Enforced	Short TTL	No
Static Security Analysis [28]	Model-based	No	No	Static	No	No
in-toto agent-decision v0.1 [2]	Decision-level	Yes	Statement	No	No	Yes
APAS [1]	Chain-level	Yes	Statement	No	Partial	Yes
AAS-1 [13]	Record-level	Yes	Custom	No	Partial	Yes
EviAssure	Merkle ($N=1M$)	Ed25519	Full Envelope	Fail-Closed	Nonce+Time	Yes

Table 4: Adversary capability classes mapped to evaluation vectors and enforcement.

Class	Vectors	Enforcement
$\mathcal{A}_1$ Trace tampering	V2, V8, V12	Merkle root re-derivation over all leaves
$\mathcal{A}_2$ Signature forgery	V1, V3	Ed25519 over canonical payload; trusted-key registry
$\mathcal{A}_3$ Replay / skew	V4, V5, V11	Nonce cache (validity window) + freshness bounds
$\mathcal{A}_4$ Key governance	V9	CRL membership; KMS ARN boundary pattern
$\mathcal{A}_5$ Malleability	V10	Canonical JSON serialization in signed payload
Policy compliance	V6, V7	Drift and pass-rate thresholds